

Evaluating GenAI Startups in Cybersecurity: An Analysis Framework

Author: Dinis Cruz

Version: 1.0

Date: March 2026

Audience: Investors, advisors, founders, and cybersecurity practitioners evaluating GenAI-enabled security companies

Purpose

This framework provides a structured approach to evaluating GenAI startups operating in the cybersecurity space, drawing on twenty years of experience in the industry. It applies equally to startups raising their first round and to growth-stage companies with significant funding, customers, and polished marketing materials.

The core thesis: many GenAI cybersecurity startups are solving the wrong problem, or solving the right problem in a way that doesn't survive the next model improvement. This framework helps distinguish between companies building durable value and companies riding a temporary wave.

The Seven Evaluation Criteria

1. Are You Solving a Structural Problem or a Temporary Mess?

The GenAI market has created two types of opportunity:

Temporary mess: Enterprises adopted GenAI rapidly. Many deployments are out of control. LLM projects have failed or caused problems — sometimes not because the LLM failed, but because the project was solving the wrong problem in the first place. There is acute demand for "control" and "governance" and "monitoring." This demand is real, but it is a symptom of the current state of affairs, not a structural need.

Structural problem: The fundamental challenges of enterprise software — context, permissions, workflows, data provenance, business rules — don't go away when you add GenAI. They get harder. Solutions that address these structural problems become more valuable as the technology matures.

The question to ask: If every enterprise had perfect visibility and control over their AI deployments tomorrow, would this company still have a product? If the answer is no, the company is selling into a temporary mess. The mess will clean itself up — through better tooling, better practices, and better models — and the company's market disappears with it.

The analogy: This is similar to early cloud security companies that sold "visibility into shadow IT." The problem was real, but it was a symptom of immature cloud adoption. As cloud platforms matured and enterprises built proper governance, the standalone "shadow IT discovery" market collapsed. The companies that survived were the ones solving deeper problems that happened to also address shadow IT.

The same pattern is playing out with "shadow AI" today.

2. Can You Deliver Value Without Mass Adoption?

Many cybersecurity products share a fundamental challenge: they require wide deployment and high coverage to deliver meaningful value. They need to be inline with a majority of traffic, a majority of endpoints, a majority of LLM calls — for their analysis to be comprehensive enough to be useful.

This creates a paradox, especially in the GenAI space: **you're selling a solution for out-of-control AI that requires the AI to be under control to work.**

The coverage problem has plagued cybersecurity for decades. Products that need to see everything before they can protect anything face an uphill battle because:

- Getting comprehensive coverage is an engineering and organisational challenge that takes years
- Shadow IT / shadow AI exists precisely because it's out of control — if you could route everything through one point, you wouldn't need the product
- The easy integrations (well-managed, sanctioned tools) aren't where the risk is — it's everything else
- As soon as you put something inline, you become a single point of failure (see Criterion 3)

The question to ask: Can this product deliver value on a single use case, for a single team, on day one? Or does it require an enterprise-wide rollout, executive sponsorship, and months of integration before anyone sees benefit?

The best products deliver immediate value at the unit level and compound as adoption grows. The worst products promise enterprise-wide visibility but deliver nothing until everything is connected.

3. What Happens When You Go Inline?

Cybersecurity has a long, painful history with inline deployments. The moment you put something in the critical path between a user and a service, you inherit a set of problems:

You become a single point of failure. If you centralise LLM traffic through your product, every outage is your outage. Every latency spike is your latency spike. Every false positive is a blocked workflow.

Context becomes king — and context is hard. What looks dangerous in one environment is completely legitimate in another. What's benign for one team is a policy violation for another. You run out of simple, obvious cases very quickly and enter the land of edge cases where every decision is a business decision, not a technical one.

You become the scapegoat. In cybersecurity, we know this intimately: 99% of the time the outage had nothing to do with the security product, but because we introduced something inline — because we added a process, a proxy, a filter — we are the first to be blamed, the first to be removed, and the first to be required to prove we didn't cause the problem.

The industry response is well-established: Most cybersecurity products end up deployed out-of-band, in monitor mode, in "watch and alert" mode — because the operational risk of inline deployment is too high. This significantly reduces their value proposition but is a pragmatic response to reality.

The question to ask: Is this product designed to be inline (intercepting, filtering, modifying LLM traffic)? If so, how do they handle false positives, latency, context-dependent decisions, and the inevitable

blame when something goes wrong? If they haven't thought deeply about this, they haven't talked to enough enterprise security teams.

4. How Are You Modelling Business Context?

This is where most GenAI security startups fall short. The signal to look for: **is this company solving business context problems, or LLM-specific technical problems?**

Prompt injection detection, toxic content filtering, PII leakage prevention — these are important, but they are technical problems that will increasingly be solved by the models themselves (or by simple, commoditised tooling). The hard, durable problem is understanding business context: workflows, state, ownership, permissions, authorisation models, data provenance, and the relationships between all of these.

The role of semantic graphs: Without a way to model relationships — between users, data, permissions, workflows, and business rules — any security product is reduced to pattern matching on individual interactions. This works for obvious cases (the low-hanging fruit) but fails for anything requiring context.

A solution that understands business context solves prompt injection "by proxy" — not by detecting injection patterns, but by ensuring that the LLM's actions are constrained by a business context that makes injection attempts irrelevant. The LLM can be tricked, but the system around it enforces the actual business rules.

The question to ask: If you ask this company "how do you handle a situation where the same prompt is dangerous in Department A but legitimate in Department B?" — do they have a good answer? If their answer involves more LLM analysis (an LLM judging another LLM's output), that's a red flag. If their answer involves a business context model that defines what's permitted where, that's a green flag.

Red flag: An LLM making decisions on top of another LLM. This is circular — how do you prevent prompt injection in the prompt injection detection layer? It adds complexity, cost, latency, and its own prompt injection surface.

Green flag: Business rules, permission models, and workflow constraints that are deterministic and don't depend on LLM interpretation.

5. When the Next Model Ships, Do You Get Better or Gain a Competitor?

This is the survivability test. GenAI is evolving faster than any technology in recent history. Every few months, new models ship with new capabilities, better safety features, improved reasoning, and broader language support.

The question to ask: When the next generation of models arrives, does this company:

(a) **Get better** — because their value is above the model layer (business context, workflows, integration, customer data), and better models make their product more capable?

(b) **Gain a competitor** — because their product fills a gap that the new model now fills natively? Their innovation from 12 months ago is now a checkbox feature in the foundation model.

Many current GenAI security products are designed to operate at the current errors, blind spots, and limitations of current models. They are essentially building products on top of attrition — betting that the models will remain limited in specific ways. This is a losing bet.

The Wardley Maps lens: If your value proposition depends on attrition occurring in your competition (which is the model providers themselves), you are in a structurally weak position. You're building on the same platform that's about to eat your market.

Recent example: When a major model provider announces a cybersecurity analyst agent, the market panics — not because the provider will destroy the cybersecurity market (other things matter too), but because they will make redundant many products that innovated a year ago and can't leverage the new capabilities.

What survives: Customer data, workflow integration, business context models, accumulated institutional knowledge, and the ability to deploy anywhere. These are moats that model improvements strengthen rather than erode.

6. Is the Open Source Strategy Real or Lock-In?

Many GenAI cybersecurity companies offer an "open source" component. The question is whether it's genuine or a lock-in mechanism.

Lock-in open source: The open source component is a small piece of the puzzle. It works, it's useful, but to get real value you need to buy the commercial product. Once you adopt the open source project, you're locked into the ecosystem. Internal debates between "what goes in the open source vs. the paid product" consistently resolve in favour of lock-in. The open source becomes a funnel, not a foundation.

The test: Can a motivated team take the open source project and, without purchasing anything, solve a real problem end-to-end? If not, the open source is a marketing strategy, not a product strategy.

Real open source: The core technology is genuinely open. The company's moat is execution — the service, the hosted offering, the enterprise features, the support, the managed deployment. Customers can always leave (reducing lock-in anxiety), but they stay because the service is better than self-hosting. This is a harder business to build but a more durable one.

Why this matters for security: In cybersecurity, trust is everything. A security product that you can't inspect, can't audit, and can't deploy in isolation creates a trust deficit. Open source solves this structurally — the code is auditable, the deployment is transparent, and the customer retains sovereignty.

Sovereignty question: If the company is not genuinely open source, what prevents them from losing sovereignty? An EU-based company can be acquired by a US entity (or vice versa). A VC-funded company's incentives can shift. Open source is the only durable guarantee that customers retain control.

7. Can You Deploy Everywhere, in Complete Isolation?

This criterion tests for architectural maturity and genuine customer-first thinking.

The question to ask: Can this product be deployed in a completely air-gapped environment with zero communication to any external service? No "phone home," no telemetry, no licence server, no API calls to the mothership?

If the answer is no, several things follow:

- **Single point of failure:** The external dependency is a failure mode the customer can't control

- **Excluded deployments:** Classified environments, regulated industries, privacy-sensitive contexts, and sovereign deployments are all off the table
- **Architecture complexity:** The product has a more complex architecture than it needs (managing connectivity, failover, caching, offline modes)
- **Trust deficit:** The customer must trust not just the product but the ongoing operation of the vendor's infrastructure

Connected to the business model: Can individual teams deploy this product with sovereignty, or does it require a centralised, top-down mandate from the security team? Security-team-driven deployments are typically compliance-motivated, which means they're fought by every other team. Developer-empowering deployments (self-service, open source, easy to try) have much higher organic adoption.

The unit economics test: Can someone spend a small amount of money with this product right now and get value? Or does the first useful deployment require a six-figure enterprise contract and a three-month implementation?

The Meta-Criteria: Agentic Architecture vs. LLMs Inline

Across all seven criteria, there is a meta-pattern that separates durable companies from fragile ones:

Fragile Pattern: LLMs Inline

The product uses LLMs as an execution engine in the critical path. Every request passes through an LLM for analysis, classification, or decision-making. This means:

- Every LLM failure, hallucination, or latency spike affects the customer
- Prompt injection in the security layer is a real and ironic risk
- Cost scales linearly with traffic (every request costs money)
- The product is as good (or as limited) as the current model generation

Durable Pattern: Agentic Development, Deterministic Execution

The product uses LLMs in the development and configuration phase — agentic workflows that help build better rules, better integrations, better business context models. But the runtime execution is deterministic: business rules, permission models, workflow constraints, and code that was generated and validated by agents but runs without LLM dependency.

This means:

- The LLM's power is leveraged for scaling the creation of rules and configurations
- The runtime is fast, predictable, and doesn't depend on LLM availability
- When models improve, the development process improves (more rules, better rules, faster)
- Prompt injection is irrelevant at runtime because there are no prompts in the execution path

The question to ask: Is the business operating in an agentic way — with AI helping humans write better code, build better rules, and understand business context — or is the AI an inline execution engine that adds complexity, cost, and prompt injection risk?

Evaluation Scorecard

#	Criterion	Red Flag	Green Flag	Score (1-5)
1	Structural vs. Temporary	Solving symptoms of current GenAI mess	Solving fundamental enterprise problems that persist as AI matures	
2	Unit Value	Requires enterprise-wide deployment for any value	Delivers value to a single team on day one	
3	Inline Risk	Product is inline with LLM traffic, single point of failure	Out-of-band, non-blocking, or inline with robust context handling	
4	Business Context	LLM analysing LLM output, no business model	Semantic graphs, permission models, workflow-aware decisions	
5	Model Survivability	Value disappears when next model ships	Value increases when models improve	
6	Open Source	Lock-in open source, can't solve real problem without paying	Genuinely open core, moat is execution not code	
7	Deployment Sovereignty	Requires phone-home, centralised deployment	Air-gappable, self-service, individual team sovereignty	
Meta	Architecture	LLMs inline as execution engine	Agentic development, deterministic runtime	

Scoring guide:

- **1:** Critical red flag — this alone may be disqualifying
- **2:** Concern — significant risk that needs addressing
- **3:** Neutral — neither strong nor weak
- **4:** Positive — demonstrates good thinking
- **5:** Exceptional — this is a genuine differentiator

Interpretation:

- **35-40:** Rare — structurally strong company with durable advantages
- **28-34:** Promising — most criteria met, addressable gaps
- **21-27:** Caution — several structural concerns
- **14-20:** High risk — likely riding a temporary wave
- **8-13:** Avoid — fundamental misalignment with market dynamics

Historical Pattern Recognition

These criteria are not novel — they map directly to patterns that have played out repeatedly in cybersecurity over the past two decades:

Era	Temporary Product	What Survived
Early firewalls	"We inspect all traffic" → couldn't handle encrypted traffic at scale	Products that understood application context, not just packets
SIEM	"We collect all logs" → drowned in noise, no context	Products that modelled relationships and business rules (SOAR, XDR)
Cloud security	"Shadow IT discovery" → the problem self-solved as cloud matured	Products that integrated into DevOps workflows (shift-left, infrastructure-as-code)
Endpoint security	"Signature-based AV" → signature evasion became trivial	Products with behavioural analysis and business context (EDR)
GenAI security (now)	"Prompt injection detection" → models will solve this themselves	Products that model business context, workflows, and permissions (?)

The pattern is consistent: **technical-only solutions get commoditised; business-context solutions endure.**

How to Use This Framework

For Investors

Apply the scorecard during due diligence. Pay particular attention to Criteria 4 (business context) and 5 (model survivability) — these are the strongest predictors of long-term value.

For Founders

Use the framework for honest self-assessment. If you score poorly on multiple criteria, it doesn't mean your company is doomed — it means you have strategic decisions to make now, while you still can.

For Enterprise Buyers

Apply the framework when evaluating vendor pitches. The vendors who can articulate strong answers to all seven criteria are the ones whose products will still be relevant in three years.

For Cybersecurity Practitioners

Use the historical pattern recognition table to calibrate your instincts. If a GenAI security product feels like it's repeating a pattern you've seen before, it probably is.

What This Framework Does NOT Evaluate

This framework assesses structural positioning and architectural durability. It does not assess:

- **Team quality** — which matters enormously but requires different evaluation methods
- **Current traction** — a company can have great traction in a temporary market
- **Financial metrics** — revenue, burn rate, and unit economics matter but are orthogonal to these criteria
- **Specific technical capabilities** — the framework is deliberately technology-agnostic
- **Go-to-market execution** — a structurally strong company can still fail on execution

These factors should be evaluated separately and combined with this structural assessment for a complete picture.

This framework will be applied to specific companies in subsequent analyses, including self-assessment of the author's own ventures.

This document is released under the Creative Commons Attribution 4.0 International licence (CC BY 4.0).